DAY 19 | HANDS-ON ACTIVITIES

Containers and Azure Deployment: Problem Statements

Seven problems. Steps that create Azure resources are optional and need permission to use a subscription; always delete them afterwards.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Twelve Factor Check

Level: Warm-up | **Concepts:** cloud-ready habits

Review your Task API and list every value that should come from an environment variable instead of code. Change at least three.

Sample run / expected result

```
DATABASE_URL, APP_ENV, CORS_ORIGINS, JWT_SECRET, APP_VERSION
```

Hints

- Search your code for URLs, keys and hard-coded ports.

Problem 2: Health and Ready

Level: Easy | **Concepts:** health endpoints, TestClient

Add /health (always 200) and /ready (200 only when the database can be reached, else 503). Test both, including a simulated database failure.

Sample run / expected result

```
/health -> 200
/ready (db down) -> 503 {"ready": false}
```

Hints

- Wrap the database ping in try/except.

Problem 3: Compose the Stack

Level: Medium | **Concepts:** docker compose

Write docker-compose.yml for the API and MongoDB with a named volume and environment variables. Start it, create a task, restart, and confirm the task is still there.

Sample run / expected result

```
docker compose up --build
POST /tasks -> 201
docker compose restart
GET /tasks -> still 1 task
```

Hints

- The API reaches Mongo at mongodb://db:27017.

Problem 4: Deploy to App Service

Level: Challenging | **Concepts:** ACR, App Service, CLI

In Cloud Shell, create a resource group and registry, build the image with az acr build, create a Linux App Service from the image, set WEBSITES_PORT=8000, and open /health.

Sample run / expected result

```
https://tasks-api-yourname.azurewebsites.net/health -> {"status": "ok"}
```

Hints

- Give the web app access to pull from ACR (managed identity with AcrPull, or enable the admin user for a quick test).
- Delete the resource group when done.

Problem 5: Deploy to Container Apps

Level: Medium | **Concepts:** az containerapp up

Deploy the same API with az containerapp up from source, then view the logs and call /health on the generated URL.

Sample run / expected result

```
Container app created. URL: https://tasks-api.<random>.centralindia.azurecontainerapps.io
```

Hints

- --ingress external --target-port 8000

Problem 6: CD Pipeline YAML

Level: Challenging | **Concepts:** stages, environments, approvals

Write a pipeline with Test, Build (Docker@2 buildAndPush to ACR, tag with Build.BuildId), DeployDev and DeployProd (deployment jobs targeting dev and production environments). Validate it with yaml.safe_load.

Sample run / expected result

```
Stages: Test -> Build -> DeployDev -> DeployProd
DeployProd environment: production
```

Hints

- Approvals are configured on the environment in the Azure DevOps UI, not in YAML.

Problem 7: Runbook

Level: Stretch | **Concepts:** operations, rollback

Write a one-page runbook for the Task API: deploy, verify, read logs, roll back to the previous image tag, rotate a secret, and clean up resources.

Sample run / expected result

```
Rollback: az containerapp update -n tasks-api -g rg-prod --image acr.azurecr.io/tasks-api:<pr
```

Hints

- Write it for a teammate who has never seen the app.

Ganbatte (頑張って): do your best